

Questão 1 [2 pontos]:

São dadas duas listas ordenadas de tamanho m e n . Dê um algoritmo de tempo $O(\log m + \log n)$ para computar o k -ésimo menor elemento da união das duas listas.

Questão 2 [2 pontos]:

Suponha que você tenha k arrays ordenados $A_1, A_2, \dots, A_k$, cada um com n elementos. Você deseja combiná-los em um único array ordenado de tamanho kn . Uma maneira de fazer isso seria usar a operação de intercalação (*merge*). Sabendo que a intercalação de dois arrays X e Y precisa de $|X| + |Y|$ passos.

1. Calcule quantos passos o algoritmo a seguir leva: Primeiro intercale os arrays A_1, A_2 , depois intercale o resultado com A_3 e assim por diante.
2. Projete um algoritmo melhor para este problema.

Questão 3 [2 pontos]:

Dado um conjunto S de strings, encontre de forma eficiente o prefixo comum mais longo. Exemplos:

Entrada: $S = \{\text{"instituto", "instigar", "instável", "instaurar"}\}$, Saída: "inst"

Entrada: $S = \{\text{"portuguese", "português", "portugal"}\}$, Saída: "portug"

Questão 4 [2 ponto]:

Considere o seguinte problema.

Entrada: Grafo não direcionado $G = (V, E)$; pesos de arestas w_e ; subconjunto de vértices $U \subset V$.

Saída: A Árvore Geradora mais leve na qual os nós de U são folhas (pode haver outras folhas na árvore também).

Questão 5 [2 ponto]:

Considere os dois algoritmos gulosos para o CAIXEIRO VIAJANTE. Mostre que ambos não são exatos, isto é, não garantem retornar a solução ótima.

Algoritmo 1 (Algoritmo do vizinho mais próximo)

1. Inicialize todos os vértices como não visitados.
2. Selecione um vértice arbitrário, defina-o como o vértice atual u . Marque u como visitado.
3. Descubra a aresta mais curta conectando o vértice atual u e um vértice não visitado v .
4. Defina v como o vértice atual u . Marque v como visitado.
5. Se todos os vértices do domínio forem visitados, então termine. Caso contrário, vá para o passo 3.

Algoritmo 2 (Algoritmo da inserção mais barata)

1. Crie um ciclo C com três vértices quaisquer.
2. Escolha um vértice k qualquer fora de C .
3. Encontre uma aresta $(i, j) \in C$ tal que $d_{ik} + d_{kj} - d_{ij}$ é mínimo.
4. Adicione k a C , remova a aresta (i, j) e adicione as arestas (i, k) e (k, j) .
5. Se C contiver todos os nós, pare. Caso contrário, vá para 2.

Questão 6 [2 pontos]:

Um ladrão entra em uma loja e vê um conjunto I de n itens, $I = \{a_1, a_2, \dots, a_n\}$. Cada item tem um peso associado w_i e um valor v_i . Idealmente, o ladrão gostaria de roubar tudo para obter o máximo benefício. No entanto, há muito que ele pode carregar. O ladrão tem uma mochila com capacidade K . O ladrão agora tem que determinar quais itens roubar para que seu peso total não exceda K e seu valor total seja máximo.

1. Suponha que os itens sejam tais que uma fração de um item possa ser retirada, ou seja, o ladrão pode pegar alguma parte de um item (por exemplo, 0,4 w_i de a_i) e deixar a parte restante. Isso é chamado de Problema da Mochila Fracionada. Seja $A \subseteq I$ o subconjunto de itens que o ladrão rouba. Elabore um algoritmo guloso $O(n \log n)$ para encontrar o subconjunto A tal que $\sum_{a_i \in A} w_i \leq K$ e $\sum_{a_i \in A} v_i$ é máximo.
2. Suponha que os itens só possam ser retirados como um todo, ou seja, o ladrão só pode pegar ou deixar um item; ele não pode pegar uma fração de um item. Isso é chamado de Problema da Mochila Binária (ou Mochila 0-1). Seu algoritmo guloso para a versão fracionária do problema (pergunta anterior) ainda encontra uma solução ótima? Justifique sua resposta.

Alg(A, B, K)

$$meio_1 = A.length / 2$$

$$meio_2 = B.length / 2$$

$$Elemento_{meio} = meio_1 + meio_2$$

$$if (meio_1 == K \text{ ou } meio_2 == K)$$

return $A[meio_1] < B[meio_2] ? A[meio_1] : B[meio_2]$

else

$$if (meio_1 < K)$$

return Alg(A[meio_1+1, ..., A.length], B, K)

else

return Alg(A, B[meio_2+1, ..., B.length], K)

$$if (meio_2 < K)$$

return Alg(A, B[meio_2+1, ..., B.length], K)

else

return Alg(A, B[meio_2+1, ..., B.length], K)

return -1

Deveria comparar

meio_1 + meio_2 com K

e
A[meio_1] com B[meio_2]

✓

2.0

2)

1 2 3 4 5

(x+y) + y + y + y + y

2 2 3 4 5 2

4

7

11

15

19

23

27

31

3. o algoritmo leva

4. A + B

5. A + B

6. A + B

7. A + B

8. A + B

9. A + B

10. A + B

$$-|A_s| + \sum_{i=0}^n |A_i| + |B_{i-1}|$$

onde B_{i-1} é o resultado da soma dos 2 termos anteriores.

Podemos usar o alg 2
seguir para melhorar este
problema

1 - faça uma operação de merge nos 2 primeiros arrays
e nos 2 ultimos continue ate que tenha feito merge em
todos os arrays 2 a 2

Ex: com 6 arrays $A_1, \dots, A_6$

✓ (1,0)

merge(A_1, A_2) e merge(A_5, A_6) e merge(A_3, A_4)

2 - realize o passo 1 ate que sobrem 3 ou 2 arrays resultantes

3 - faça um merge nos arrays resultantes restantes.

nº de passos = $n \cdot k$ mais eficiente

X $\rightarrow O(k \log k)$

6) 1) 1 - descubra o valor por peso de cada objeto, pode ser dividindo
valor/peso total e os separe em grupos correspondentes

2 - ordene os objetos pelo valor por peso achado no passo 1
 $O(n \log n)$

3 - faça uma escolha gulosa pegando primeiramente os item com
o maior valor por peso ate que não aça mais espaço na mochila
dessa forma o custo sera basicamente o da ordenação $O(n \log n)$

2) X

Pseudo-código

(1,0)

12) mochila (itens, k, mochila)

Sort(itens) // pelo preço

Enquanto K possuir capacidade

item = itens.pop()

se $(K - \text{item.peso}) > 0$

! mochila.add(item)

return mochila.

6.2) ?

não o algoritmo da questão anterior não conseguiria retornar uma solução ótima. Podemos mostrar isso por contra exemplo:

Suponha que o ladrão tem uma mochila com $K=4$ e ele pode escolher roubar entre os itens $i_1(\text{valor}, \text{peso})$

$i_1 = (30, 2)$, $i_2 = (9, 3)$

1,0

Se ele não puder pegar uma fração do item o algoritmo resultaria com a mochila contendo apenas o item i_1 , e ainda sobaria 2 de espaço na mochila. Ele roubaria 30 unidades de valor no total. mas se pudesse fracionar ele poderia acrescentar $\frac{2}{3}$ de i_2 e teria i_1 e $\frac{2}{3}$ de i_2 esgotando a capacidade da mochila mas roubaria um total de 36 unidades de valor no total: $30 + \frac{2}{3} \cdot 9 = 36$

é o algoritmo acima?

3) maior prefixo (S) // onde cada S_i pode ser dado por S_{i-1} + char

110000 de simplificação

PreS = ""

- Sort(S) // ordene S pelo tamanho de S // $O(n \log n)$

- K = S[3].size

- for i ← 1 → K // $O(K)$

 if (igual(S, S[i], i)) // S[i] é a letra de pos i de S[3]

 PreS = PreS + S[i] // $O(1)$

 else break

return PreS

igual(S, char c, int i)

Para cada $S_i \in (S - \{S[3]\})$ // $O(n-3)$

 if $S_i[i] != c$

 return false

return true

$T(n) = n \log n + K \cdot (n-3)$

$\therefore O(n \log n)$

2,5

Ineficiente

Poderia fazer em $O(Kn)$

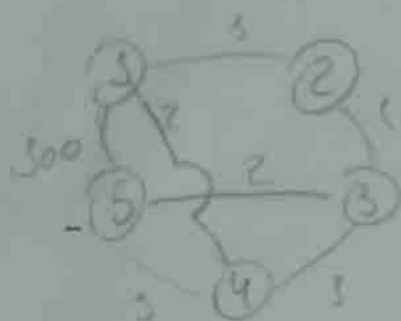
1) ache uma ABM para um $G' = G - U$, ou seja uma ABM para um subgrafo de G que não contenham os nos $U \in V$, pode ser utilizado o algoritmo de prim ou Kruskal neste passo

2) com a ABM de G' ligue os vertices pertencentes a U de forma que a ligação seja feita pela aresta de menor custo que o ligue a algum vertex que pertence a ABM em questão, este passo pode ser feito de forma gulosa. Como?

3) por fim, após 1 e 2 temos o que se é pedido.

Pseudo-código 2,0

3) Alg 1. por contra exemplo temos
Suponha um grafo G:



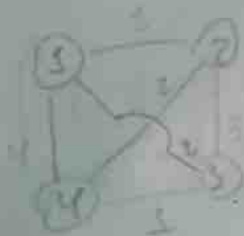
utilizando o Algoritmo 1 obteríamos o seguinte caminho escolhendo o vértice 1 por exemplo, sem perda de generalidade

$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 1$, com peso = 804

podem pensar que a solução ótima seria

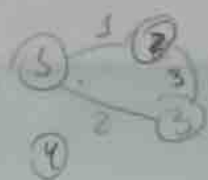
$1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 1$, com peso = 6

Alg 2. por contra exemplo temos
considere um grafo G:



pelo algoritmo 2 obteríamos:

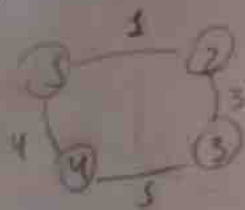
② criando $C = (1,2), (2,3), (1,3)$



④ - o vértice 4 é a única escolha possível então removemos usando a aresta (1,3):

$$d_{14} + d_{43} - d_{13} = 4 + 1 - 2 = 3$$

⑤ realizando o passo 4 do algoritmo



Peso = 3

perem a solução ótima seria.



Peso = 6

X

0,5

mas (2,3) é mínimo

$$d_{42} + d_{43} - d_{23} = 2 + 1 - 3 = 0$$